TecPro

TECNOLOGIAS

Desenvolvimento, Apostilas e Cursos Online



Linguagem



Listas



Árvores



Complexidade

Algoritmos e Estruturas de Dados

Em C++, com comparações em Python

Algoritmos e Estruturas de Dados

Em C++, com comparações em Python

TecPro Tecnologias

Desenvolvimento, Apostilas e Cursos Online

2026

Todos os direitos reservados. Nenhuma parte desta publicação pode ser reproduzida, armazenada ou transmitida de qualquer forma, ou por qualquer meio, sem autorização prévia da TecPro Tecnologias.

Sumário

Capítulo 1 – Fundamentos de C++ para Estruturas de Dados **4**
1.1 Por que estudar estruturas de dados em C++? 4
1.2 Estrutura de um programa em C++ 4
1.3 Tipos, variáveis e vetores 5
1.4 Funções e passagem de parâmetros 6
Por valor: a função recebe uma cópia 6
Por referência: a função trabalha na variável original 6
Por referência constante: sem cópia e sem alteração 7
1.5 Ponteiros 8
1.6 Memória dinâmica: new e delete 9
1.7 Structs: agrupando dados 10
Uma prévia: o nó de uma lista encadeada 10
1.8 Classes: dados e comportamento juntos 11
1.9 Templates: estruturas para qualquer tipo 13
Erros Comuns 13

Capítulo 1 – Fundamentos de C++ para Estruturas de Dados

Em Algoritmos I você aprendeu a base da programação: variáveis, tipos e operadores, condicionais e laços, funções, strings, vetores e matrizes. Também conheceu, de forma introdutória, listas, pilhas, filas, busca, ordenação e recursão. Agora vamos aprofundar: construir essas estruturas por dentro, de forma eficiente, e analisar o custo de cada operação.

Para isso, você precisa de algumas ferramentas do C++ que as estruturas de dados usam o tempo todo. Este capítulo faz duas coisas:

- **retoma rapidamente** o que você já conhece, agora escrito em C++: estrutura de um programa, tipos, vetores e funções (seções 1.2 a 1.4);
- **apresenta o que é novo** e indispensável daqui em diante: ponteiros, memória dinâmica, structs, classes e templates (seções 1.5 a 1.9).

Se você já se sente seguro com a primeira parte, pode passar por ela rapidamente. A segunda parte merece atenção total: é a base de todos os capítulos seguintes.

1.1 Por que estudar estruturas de dados em C++?

Em linguagens como Python, Java ou JavaScript, a memória é administrada automaticamente. Você cria uma lista, adiciona elementos e nunca precisa pensar onde eles estão guardados. Isso é ótimo para produtividade, mas esconde justamente o que queremos entender nesta disciplina: **como os dados são organizados na memória**.

Em C++, você enxerga e controla essa organização. Uma lista encadeada deixa de ser uma caixa-preta e passa a ser o que ela realmente é: um conjunto de blocos de memória, cada um apontando para o próximo. Quando você entende isso, passa a entender também por que uma operação é rápida ou lenta, e esse é o centro da análise de algoritmos.

Ao longo do livro, sempre que ajudar a entender um conceito, vamos mostrar como a mesma ideia aparece em Python. Assim você vê as duas faces da moeda: o que acontece *por dentro* (C++) e como isso costuma ficar *escondido* em uma linguagem de mais alto nível (Python).

1.2 Estrutura de um programa em C++

Todo programa em C++ começa a ser executado pela função `main`. Veja um programa completo:

```
#include <iostream>   // biblioteca de entrada e saída (cin, cout)
using namespace std;  // permite escrever cout em vez de std::cout

int main() {
    int idade;
    cout << "Digite sua idade: ";
    cin >> idade;                       // lê um valor do teclado
```

```
    cout << "Daqui a 10 anos voce tera " << idade + 10 << " anos" <<
    endl;
    return 0;                              // 0 indica que terminou sem
    erro
}
```

Alguns pontos importantes:

- `#include` traz uma biblioteca para o programa. A `iostream` oferece `cin` (entrada) e `cout` (saída).
- Todo comando termina com `;` e todo bloco fica entre `{` e `}`.
- C++ diferencia letras maiúsculas de minúsculas: `idade` e `Idade` são variáveis diferentes.
- O programa precisa ser **compilado** antes de rodar. Com o compilador g++, por exemplo: `g++ programa.cpp -o programa`.

Comparando com Python

Em Python, o mesmo programa não precisa de `main`, de tipos nem de compilação, porque é interpretado linha a linha:

```
idade = int(input("Digite sua idade: "))
print(f"Daqui a 10 anos voce tera {idade + 10} anos")
```

Em C++, o compilador verifica os tipos antes de o programa rodar. Muitos erros que em Python só aparecem durante a execução, em C++ aparecem já na compilação.

1.3 Tipos, variáveis e vetores

Uma variável é um espaço na memória com um nome e um tipo. O tipo define quanto espaço ela ocupa e que valores pode guardar.

Tipo	Guarda	Exemplo
`int`	números inteiros	`int qtd = 10;`
`double`	números com casas decimais	`double nota = 8.5;`
`char`	um caractere	`char letra = 'A';`
`bool`	verdadeiro ou falso	`bool ok = true;`
`string`	texto (precisa de `<string>`)	`string nome = "Ana";`

Um **vetor** (array) guarda vários valores do mesmo tipo, um ao lado do outro na memória. Os índices começam em 0:

```
int notas[5] = {7, 9, 6, 10, 8};   // 5 inteiros seguidos na memória

for (int i = 0; i < 5; i++) {
    cout << "Nota " << i << ": " << notas[i] << endl;
}
```

O vetor tem uma limitação que vai motivar vários capítulos deste livro: o **tamanho é fixo**. Se você declarou espaço para 5 notas e chegou uma sexta, não há onde colocá-la. Inserir um elemento no meio também é trabalhoso, porque todos os seguintes precisam ser deslocados uma posição. As estruturas que vamos construir, como as listas encadeadas, existem justamente para contornar essas limitações.

1.4 Funções e passagem de parâmetros

Uma função é um trecho de código com nome, que recebe dados (parâmetros), faz um trabalho e pode devolver um resultado. Funções evitam repetir código e deixam o programa mais fácil de ler e de manter.

```
// recebe os três coeficientes e diz se a equação tem raízes reais
bool temRaizReal(double a, double b, double c) {
    double delta = b * b - 4 * a * c;
    return delta >= 0;
}

// void: a função não devolve valor, só executa uma ação
void imprimirVetor(int v[], int tamanho) {
    for (int i = 0; i < tamanho; i++) {
        cout << v[i] << " ";
    }
    cout << endl;
}
```

O ponto mais importante para este livro é **como os parâmetros chegam à função**. Em C++ há três formas.

Por valor: a função recebe uma cópia

```
void dobrar(int x) {
    x = x * 2;          // altera só a cópia
}

int main() {
    int n = 5;
    dobrar(n);
    cout << n << endl;  // imprime 5: o n original não mudou
}
```

Por referência: a função trabalha na variável original

Basta acrescentar `&` depois do tipo do parâmetro. A função passa a usar a própria variável de quem chamou, sem cópia:

```
void dobrar(int &x) {
    x = x * 2;          // altera a variável original
}
```

```
void trocar(int &a, int &b) {
    int auxiliar = a;
    a = b;
    b = auxiliar;
}

int main() {
    int n = 5;
    dobrar(n);
    cout << n << endl;  // imprime 10

    int p = 1, q = 2;
    trocar(p, q);
    cout << p << " " << q << endl;  // imprime 2 1
}
```

Por referência constante: sem cópia e sem alteração

Quando o dado é grande (um texto longo, um vetor de milhares de itens) e a função só precisa ler, use `const &`. Você evita a cópia, que custa tempo e memória, e o compilador garante que a função não altera o original:

```
#include <string>

int contarVogais(const string &texto) {
    int total = 0;
    for (char c : texto) {
        if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u')
        {
            total++;
        }
    }
    return total;
}
```

Comparando com Python

Python não deixa você escolher. Números e textos se comportam como se fossem passados por valor, porque não podem ser alterados (são *imutáveis*). Já listas e dicionários são passados de forma que a função pode modificá-los:

```
def dobrar(x):
    x = x * 2          # cria um novo número só dentro da função

def adicionar(lista):
    lista.append(99)   # modifica a lista original

n = 5
dobrar(n)
```

```
print(n)       # 5

numeros = [1, 2]
adicionar(numeros)
print(numeros) # [1, 2, 99]
```

Em C++ essa decisão é sua e fica escrita na assinatura da função, o que torna o comportamento explícito.

1.5 Ponteiros

Toda variável ocupa uma posição na memória, e cada posição tem um **endereço**, como o número de uma casa em uma rua. Um **ponteiro** é uma variável que guarda um endereço: em vez de guardar um valor, ele aponta para onde o valor está.

Dois operadores fazem todo o trabalho:

- `&variavel` devolve o **endereço** da variável;
- `*ponteiro` acessa o **valor** que está no endereço guardado (isso se chama *desreferenciar*).

```
int idade = 20;
int *p = &idade;      // p guarda o endereço de idade

cout << idade << endl;  // 20
cout << p << endl;      // um endereço, por exemplo 0x61fe14
cout << *p << endl;     // 20: o valor que está no endereço

*p = 21;                // altera idade através do ponteiro
cout << idade << endl;  // 21
```

Uma forma de visualizar o que aconteceu na memória:

Variável	Endereço	Conteúdo
`idade`	`0x61fe14`	`21`
`p`	`0x61fe18`	`0x61fe14` (aponta para `idade`)

Um ponteiro que não aponta para nada deve valer `nullptr`. Você vai usar muito esse valor: é ele que marca o fim de uma lista encadeada ou um galho vazio de uma árvore. Antes de desreferenciar um ponteiro, verifique se ele não é nulo:

```
int *q = nullptr;   // não aponta para nada

if (q != nullptr) {
    cout << *q << endl;  // só acessa se for seguro
}
```

Referência ou ponteiro?

Os dois permitem trabalhar com a variável original, mas têm diferenças. Uma **referência** (`int &r = x;`) é um apelido: precisa nascer ligada a uma variável e nunca muda de alvo. Um **ponteiro** pode começar vazio (`nullptr`) e pode passar a apontar para outro lugar. Por isso as estruturas de dados usam ponteiros: um nó de uma lista precisa poder apontar para *nenhum* nó, ou mudar de vizinho quando você insere ou remove elementos.

1.6 Memória dinâmica: new e delete

As variáveis declaradas dentro de uma função ficam em uma área chamada **pilha de execução** (*stack*). Elas são criadas quando a função começa e destruídas automaticamente quando ela termina. Isso é prático, mas tem um limite: você não consegue criar, durante a execução, quantos elementos quiser e mantê-los vivos depois que a função acabar.

Para isso existe a memória dinâmica, ou **heap**. Com `new`, você pede ao sistema um espaço que continua existindo até você mesmo liberá-lo com `delete`:

```
int *p = new int;     // reserva espaço para um int no heap
*p = 42;
cout << *p << endl;   // 42
delete p;             // devolve o espaço
p = nullptr;          // boa prática: não deixar o endereço antigo

// vetor com tamanho decidido durante a execução
int n;
cout << "Quantos valores? ";
cin >> n;
int *valores = new int[n];   // n inteiros no heap
for (int i = 0; i < n; i++) {
    valores[i] = i * i;
}
delete[] valores;            // vetor: libera com delete[]
```

Toda estrutura de dados dinâmica deste livro funciona assim: cada novo elemento é criado com `new` e, quando sai da estrutura, é liberado com `delete`.

Regra de ouro da memória dinâmica

Para cada `new` deve existir exatamente um `delete` (e para cada `new[]`, um `delete[]`). Se você esquecer o `delete`, o espaço fica preso até o programa terminar: é o chamado **vazamento de memória** (*memory leak*). Se fizer `delete` duas vezes, ou usar o ponteiro depois do `delete`, o comportamento é imprevisível e o programa pode travar.

Comparando com Python

Em Python não existe `delete` para liberar memória. O próprio interpretador percebe quando um objeto não é mais usado por ninguém e o remove: é a **coleta de lixo** (*garbage collection*). Isso evita vazamentos e ponteiros inválidos, mas tem um custo:

você perde o controle de quando a memória é liberada e de quanto espaço cada estrutura realmente ocupa.

1.7 Structs: agrupando dados

Uma `struct` junta vários dados relacionados em um único tipo novo. Em vez de ter variáveis soltas para nome, idade e matrícula de um aluno, você cria o tipo `Aluno`:

```
struct Aluno {
    string nome;
    int matricula;
    double media;
};

int main() {
    Aluno a;
    a.nome = "Ana";          // o ponto acessa cada campo
    a.matricula = 2026001;
    a.media = 8.7;

    Aluno *p = &a;
    cout << p->nome << endl; // com ponteiro, usa-se a seta ->
}
```

Repare no operador `->`: quando você tem um **ponteiro** para uma struct, `p->nome` é o mesmo que `(*p).nome`. Ele vai aparecer em praticamente todo o código dos próximos capítulos.

Uma prévia: o nó de uma lista encadeada

Uma struct pode ter um campo que é ponteiro para o próprio tipo. Essa ideia simples é a base das listas encadeadas e das árvores:

```
struct No {
    int valor;
    No *proximo;   // aponta para o próximo nó (ou nullptr)
};

int main() {
    // cria três nós no heap
    No *a = new No{10, nullptr};
    No *b = new No{20, nullptr};
    No *c = new No{30, nullptr};

    // liga os nós: a -> b -> c
    a->proximo = b;
    b->proximo = c;

    // percorre a cadeia a partir do primeiro nó
    for (No *atual = a; atual != nullptr; atual = atual->proximo) {
```

```
        cout << atual->valor << " ";   // 10 20 30
    }
    cout << endl;

    delete a;
    delete b;
    delete c;
}
```

Os três nós podem estar em qualquer lugar da memória, não precisam estar lado a lado como num vetor. O que os mantém em ordem são os ponteiros. Por isso, para inserir um valor no meio, basta mudar dois ponteiros, sem deslocar nenhum elemento. No Capítulo 3, essa ideia vira uma lista completa.

1.8 Classes: dados e comportamento juntos

Uma classe vai além da struct: ela reúne os dados e as funções que trabalham com eles, e controla quem pode acessar o quê. Esse controle se chama **encapsulamento**. Os dados ficam em `private`, protegidos de alterações indevidas, e o acesso acontece por funções públicas, os **métodos**.

Como exemplo, veja uma classe que resolve o problema do tamanho fixo dos vetores: um vetor que cresce sozinho quando enche.

```
class VetorDinamico {
private:
    int *dados;        // vetor no heap
    int tamanho;       // quantos elementos estão em uso
    int capacidade;    // quantos cabem antes de crescer

    void crescer() {
        capacidade = capacidade * 2;
        int *novo = new int[capacidade];
        for (int i = 0; i < tamanho; i++) {
            novo[i] = dados[i];      // copia os elementos antigos
        }
        delete[] dados;              // libera o vetor antigo
        dados = novo;
    }

public:
    // construtor: executado quando o objeto é criado
    VetorDinamico() {
        capacidade = 2;
        tamanho = 0;
        dados = new int[capacidade];
    }

    // destrutor: executado quando o objeto deixa de existir
    ~VetorDinamico() {
        delete[] dados;
```

```
    }

    void adicionar(int valor) {
        if (tamanho == capacidade) {
            crescer();
        }
        dados[tamanho] = valor;
        tamanho++;
    }

    int obter(int indice) const {
        return dados[indice];
    }

    int quantidade() const {
        return tamanho;
    }
};

int main() {
    VetorDinamico v;
    for (int i = 1; i <= 10; i++) {
        v.adicionar(i * 10);   // cresce sozinho: 2, 4, 8, 16 posições
    }
    for (int i = 0; i < v.quantidade(); i++) {
        cout << v.obter(i) << " ";
    }
    cout << endl;
}   // aqui o destrutor libera a memória automaticamente
```

Repare em três ideias que vão se repetir ao longo do livro:

- **Construtor** (`VetorDinamico()`): prepara o objeto quando ele nasce. Tem o mesmo nome da classe e não tem tipo de retorno.
- **Destrutor** (`~VetorDinamico()`): libera os recursos quando o objeto deixa de existir. Assim, quem usa a classe não precisa se lembrar de chamar `delete`.
- **Métodos** `const`: `obter` e `quantidade` apenas consultam o objeto, e o `const` garante que não alteram nada.

Todas as estruturas dos próximos capítulos (lista, pilha, fila, árvore) serão classes com esse formato: dados privados, métodos públicos para as operações e um destrutor que libera os nós.

Comparando com Python

A lista do Python (`list`) funciona exatamente como o `VetorDinamico`: por dentro, ela reserva espaço sobrando e, quando enche, cria um espaço maior e copia os elementos. Você só não vê esse trabalho. A mesma classe em Python ficaria assim:

```
class Contador:
    def __init__(self):        # construtor
```

```
        self._total = 0        # o _ indica "uso interno"

    def incrementar(self):
        self._total += 1

    def valor(self):
        return self._total
```

Em Python, o `private` é só uma convenção (o sublinhado), e não existe destrutor para liberar memória, porque a coleta de lixo cuida disso.

1.9 Templates: estruturas para qualquer tipo

O `VetorDinamico` só guarda `int`. Se você precisar de um vetor de `double` ou de `string`, teria de copiar a classe inteira trocando o tipo. Os **templates** resolvem isso: você escreve o código uma vez, com um tipo genérico `T`, e o compilador gera a versão de cada tipo usado.

```
// função genérica: funciona com int, double, string...
template <typename T>
T maior(T a, T b) {
    return (a > b) ? a : b;
}

// struct genérica: o nó pode guardar qualquer tipo
template <typename T>
struct NoGenerico {
    T valor;
    NoGenerico<T> *proximo;
};

int main() {
    cout << maior(3, 7) << endl;                          // 7
    cout << maior(2.5, 1.5) << endl;                      // 2.5
    cout << maior(string("ana"), string("bia")) << endl;  // bia

    NoGenerico<string> *n = new NoGenerico<string>{"oi", nullptr};
    cout << n->valor << endl;
    delete n;
}
```

A biblioteca padrão do C++ (STL) já traz estruturas prontas feitas com templates, como `vector`, `list`, `stack`, `queue` e `map`. Neste livro vamos construir as nossas versões, porque o objetivo é entender como elas funcionam por dentro. Depois de entender, use as da STL nos seus projetos: são testadas e otimizadas.

Erros Comuns

- **Usar um ponteiro não inicializado.** Declarar `int *p;` e depois fazer `*p = 5;` escreve em um endereço aleatório. Inicialize sempre: com um endereço válido

ou com `nullptr`.

- **Desreferenciar `nullptr`.** Acessar `p->valor` quando `p` é nulo derruba o programa. Em listas e árvores, confira se o ponteiro não é nulo antes de usá-lo.
- **Esquecer o `delete`.** Cada `new` sem o `delete` correspondente é memória perdida. Em classes, libere no destrutor.
- **Usar `delete` em vez de `delete[]`.** O que foi criado com `new[]` deve ser liberado com `delete[]`.
- **Usar o ponteiro depois do `delete`.** O espaço já foi devolvido, e o endereço antigo não é mais seu. Depois de liberar, faça `p = nullptr;`.
- **Esperar que uma função altere um parâmetro passado por valor.** Se a função precisa modificar a variável de quem chamou, use referência (`&`) ou ponteiro.
- **Acessar um vetor fora dos limites.** Em um vetor de 5 posições, os índices vão de 0 a 4. C++ não avisa quando você acessa `v[5]`: o programa simplesmente lê ou estraga outra área da memória.